How SpiceMCP compares

Three circuits, three ways of designing them, one independent checker. SpiceMCP 1.1.0, ngspice 42, 9 Nov 2026.

The short answer

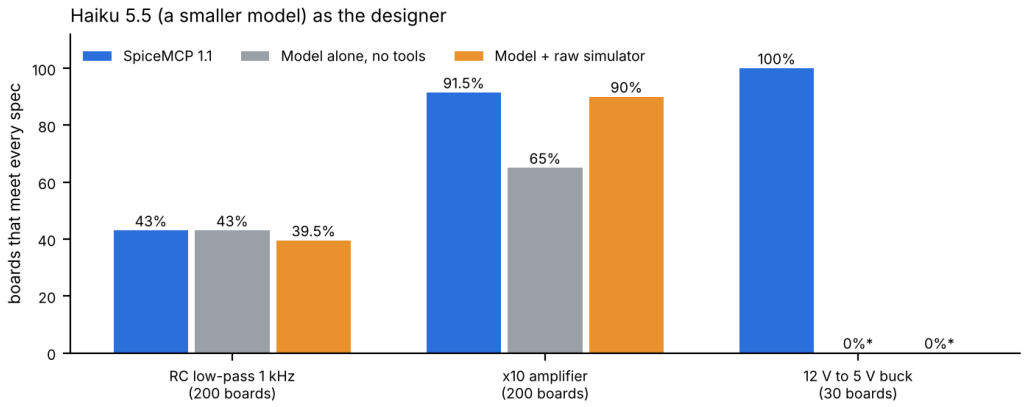
• **With a smaller model (Haiku 5.5), SpiceMCP is the only way the buck converter worked.** Haiku alone and Haiku with a raw simulator both produced bucks that miss 5 V ±2% or the ripple limit (0 of 30 boards); Haiku driving SpiceMCP got 30 of 30.

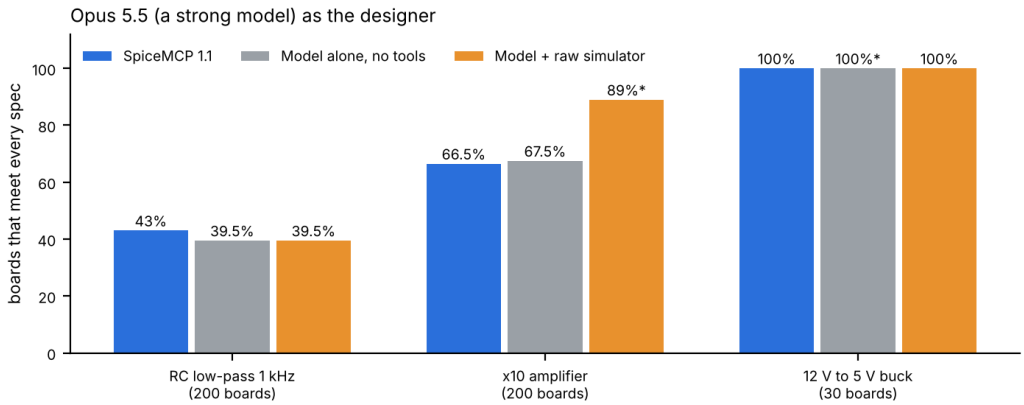
• **With a strong model (Opus 5.5), all three approaches met every spec at nominal on all three circuits.** For simple, well-known circuits a strong model already writes the textbook answer, and SpiceMCP did not beat it on pass rate or yield.

• **The RC filter caps out near 40% for everyone:** ±2% cutoff with 5% capacitors is not buildable reliably. Both SpiceMCP runs said so and named the fix (tighter capacitor).

• **Amplifier:** Haiku with SpiceMCP reached 91.5% yield by re-optimising to a centred pair (102k / 11.3k); the scripted Opus run took SpiceMCP's first answer and got 66.5%. Haiku alone got 65%.

• **What SpiceMCP costs:** it is the slowest (buck yield check about 14 min) and Haiku made 44 tool calls, 23 of them repeated 'still running' polls. Haiku with a raw simulator used 4 times the output tokens (146k vs 35k) and still failed the buck.





Yield: share of randomly built boards (resistors 1%, capacitors 5%, inductors 20%, supply ±5%) that meet every spec at −40, 25 and 85 °C. Same random boards for every arm. * = uses a value you cannot buy or an ideal switch.

What was compared

• **Two designers:** Opus 5.5 (strong) and Haiku 5.5 (smaller). Opus checked every result and wrote the report.

• **SpiceMCP 1.1:** with Opus, a script made the calls a model would make (list templates, start, optimise 20 rounds, finish, report, draw) over the real MCP connection. With Haiku, the model chose and made every call itself.

• **Model alone, no tools:** a fresh model given only the spec text; it wrote the three netlists from memory, without simulating.

• **Model + raw simulator:** a fresh model allowed only to write netlists and run ngspice, with no templates, optimiser, value snapping or yield. Budget about 25 runs.

• **Textbook reference:** for the RC filter and the amplifier the textbook answer (15.8 kΩ with 10 nF; 9.09 kΩ over 1 kΩ) is exactly what the model alone wrote, so it is not a separate column.

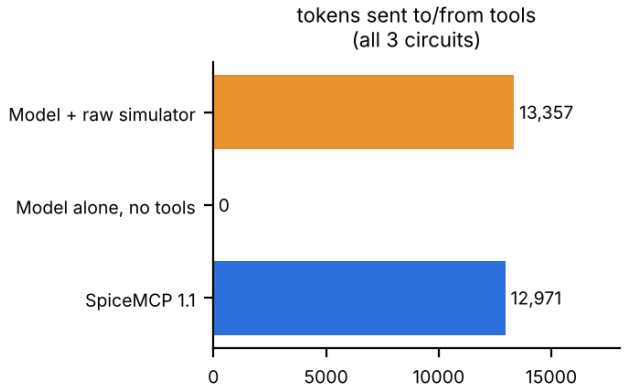
• **Other SPICE servers:** installing third-party packages was blocked in this sandbox, so they were compared on features from their public pages, not run.

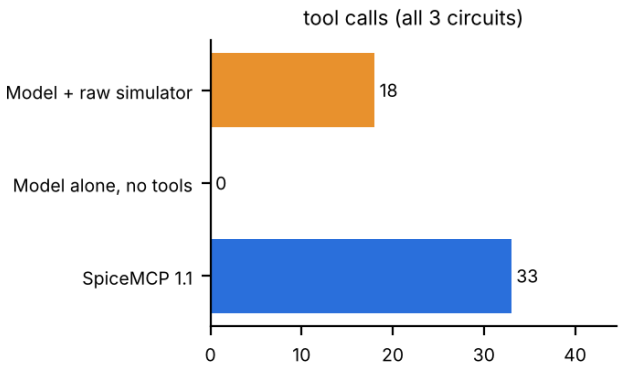
• **The checker** (measure/checker.py) is written separately from SpiceMCP, strips every arm's own analyses and measures all of them with the same decks.

Results, circuit by circuit (Opus 5.5 as the designer)

circuit	arm	values	meets spec (nominal)	measured	yield	all values buyable
RC low-pass 1 kHz	SpiceMCP 1.1	R1 88.7k, C1 1.8n	yes	fc 997 Hz	43% of 200	yes
RC low-pass 1 kHz	Model alone, no tools	R1 15.8k, C1 10n	yes	fc 1007 Hz	39.5% of 200	yes
RC low-pass 1 kHz	Model + raw simulator	R1 1.58k, C1 100n	yes	fc 1007 Hz	39.5% of 200	yes
x10 amplifier	SpiceMCP 1.1	R2 90.9k, R1 10k	yes	20.074 dB, BW 1.12 MHz, overshoot 0%	66.5% of 200	yes
x10 amplifier	Model alone, no tools	Rf 9.09k, Rg 1k	yes	20.072 dB, BW 1.10 MHz, overshoot 0%	67.5% of 200	yes
x10 amplifier	Model + raw simulator	Rf 9.09k, Rg 1.01k	yes	19.994 dB, BW 1.11 MHz, overshoot 0%	89% of 200	no (Rg)
12 V to 5 V buck	SpiceMCP 1.1	5 R/C/L	yes	11.4 V: 4.991 V / 32 mV, 12 V: 4.992 V / 35 mV, 12.6 V: 4.991 V / 35 mV	100% of 30	yes
12 V to 5 V buck	Model alone, no tools	9 R/C/L	yes	11.4 V: 5.000 V / 3 mV, 12 V: 5.000 V / 4 mV, 12.6 V: 5.000 V / 3 mV	100% of 30	yes; ideal switch
12 V to 5 V buck	Model + raw simulator	13 R/C/L	yes	11.4 V: 4.999 V / 11 mV, 12 V: 5.001 V / 15 mV, 12.6 V: 4.998 V / 14 mV	100% of 30	yes

Effort with Opus 5.5: time, calls and tokens





arm	wall-clock (3 circuits)	tool calls	simulator runs	tokens to/from tools	model output tokens	what you get back
SpiceMCP 1.1	13 min 53 s (RC 2 s, amp 3 s, buck 13.8 min), no model thinking included	33	about 1,100 (optimiser, stability and noise decks, yield boards at 3 tempera tures)	12,971	none: a script made the calls in this run	standard-value parts list (CSV), yield with confidence, part ratings to buy, schematic, report with plots
Model alone, no tools	61 s	0	0	0	5,868 (mostly reasoning)	3 netlists and one line of reasoning each
Model + raw simulator	3 min 18 s	18	37	13,357	15,329 (mostly reasoning)	3 netlists and its own corner table; no parts list, no yield, no drawing

Tokens counted with the same tokenizer for all arms (±25% against today's models). Billed input if nothing is cached, with each turn resending the conversation: SpiceMCP 197k (its 3,006-token tool list goes out every turn, and the buck needed 13 finish_design polls), model + raw simulator 145k, model alone about 1k. Model output tokens are from the API.

Against other open-source SPICE servers (features, from their public pages)

	SpiceMCP 1.1	ltspice-mcp 0.8 (Cognitohazard)	mcp-spice 0.1 (zesun33)	spicebridge
runs ngspice / LTspice	both	both	ngspice	ngspice
ready-made circuits sized by equations	13	no	no	not stated
optimises part values	yes	no (API usable by one)	no	not stated
snaps to buyable values	E12/E24/E96	no	no	not stated
Monte Carlo / yield	yes, with confidence bound	yes	no	listed
temperature and supply corners	yes	corners mentioned	no	worst-case listed
parts list and ratings to buy	yes	no	no	no
draws the schematic	yes, auto-placed	edits .asc	no	no
MCP tools	14	8	2	not checked

From each project's public page on 9 Nov 2026 (ltspice-mcp: PyPI page; mcp-spice: Glama listing; spicebridge: search listing only, its page did not load). Not run here: installing third-party packages was blocked in this sandbox. Treat 'not stated' as unknown, not as missing.

Same test with a smaller model: Haiku 5.5

• Same spec text, same parts file, same checker and the same random boards as the Opus run. Every design call was made by Haiku 5.5 (checked in the transcripts); Opus only checked the results and wrote this report.

• SpiceMCP arm: Haiku saw the real tool list and called the real server through a small shell bridge (measure/bridge.py) that keeps one MCP session open and logs every call. It chose its own calls: templates, re-optimising, finish, simulate.

• Raw-simulator arm: Haiku wrote netlists and ran ngspice itself (22 runs). No-tool arm: Haiku wrote the netlists from memory.

Results, circuit by circuit (Haiku 5.5 as the designer)

circuit	arm	values	meets spec (nominal)	measured	yield	all values buyable
RC low-pass 1 kHz	SpiceMCP 1.1	R1 13.3k, C1 12n	yes	fc 997 Hz	43% of 200	yes
RC low-pass 1 kHz	Model alone, no tools	R1 59k, C1 2.7n	yes	fc 999 Hz	43% of 200	yes
RC low-pass 1 kHz	Model + raw simulator	R1 1.58k, C1 100n	yes	fc 1007 Hz	39.5% of 200	yes
x10 amplifier	SpiceMCP 1.1	R2 102k, R1 11.3k	yes	20.019 dB, BW 1.12 MHz, overshoot 0%	91.5% of 200	yes
x10 amplifier	Model alone, no tools	Rg 1k, Rf 9.09k	yes	20.072 dB, BW 1.10 MHz, overshoot 0%	65% of 200	yes
x10 amplifier	Model + raw simulator	Rg 10k, Rfa 88.7k, Rfb 1.21k	yes	19.988 dB, BW 1.13 MHz, overshoot 0%	90% of 200	yes
12 V to 5 V buck	SpiceMCP 1.1	5 R/C/L	yes	11.4 V: 4.991 V / 32 mV, 12 V: 4.991 V / 34 mV, 12.6 V: 4.991 V / 35 mV	100% of 30	yes
12 V to 5 V buck	Model alone, no tools	9 R/C/L	no	11.4 V: 4.774 V / 54 mV, 12 V: 4.852 V / 50 mV, 12.6 V: 4.924 V / 40 mV	0% of 30	yes; ideal switch
12 V to 5 V buck	Model + raw simulator	10 R/C/L	no	11.4 V: 5.957 V / 547 mV, 12 V: 5.855 V / 511 mV, 12.6 V: 5.774 V / 477 mV	0% of 30	yes; ideal switch

Effort with Haiku 5.5

arm	wall-clock (3 circuits)	tool calls	simulator runs	tokens to/from tools	model output tokens	what you get back
SpiceMCP 1.1	21 min (16 min waiting on the buck finish)	44	about 3,900 inside SpiceMCP	25,926	34,994	standard values, yield with confidence, parts list, ratings, schematic; flagged the RC filter as failing
Model alone, no tools	4 min	0	0	0	35,157	3 netlists; 'high confidence' on RC and amp, 'low-medium' on the buck
Model + raw simulator	16 min	30	22	34,174	146,366	3 netlists; said honestly that its buck does not meet spec

Billed input if nothing is cached: Haiku + SpiceMCP about 900k, Haiku + raw simulator about 515k (both grow with every call; with prompt caching the real cost is a fraction). Tool tokens are requests plus answers through the tool.

Brutally honest reading

• With a strong model, this benchmark does not show that SpiceMCP designs better circuits. On these three it designs about the same circuits; its value is the measured yield, standard values, ratings and drawings.

• With a smaller model it clearly does: the closed-loop buck is out of Haiku's reach without it (two different failed designs: one settles slowly and sits 0.1 to 0.2 V low, one is 0.8 to 1 V high with 0.5 V ripple), and in reach with it.

• Haiku also stayed honest with SpiceMCP: it reported the RC filter as not meeting spec at 41% yield instead of claiming a pass. Haiku alone gave itself 'high confidence' on RC and amp and only 'low-medium' on the buck, which failed.

• Both no-tool bucks and the Haiku raw-simulator buck used an ideal switch, which is not a part you can buy. The checker does not punish idealisations; a stricter benchmark should.

• All arms were handed SpiceMCP's own working op-amp and comparator models. That removes the hardest part of the buck for the baselines, so the buck result still flatters them.

• The Opus SpiceMCP arm was driven by a script, not a model; the Haiku SpiceMCP arm was driven by the model itself through a bridge to the real server. The Haiku run is the fairer test of the tool.

What to fix next

• **finish_design should centre ratios, not just snap each value:** search standard-value pairs for the one closest to the middle of the limits. On the amplifier that lifts yield from 66.5% to 91%.

• **Do not shrink parts to the edge by default:** the 'smallest capacitor / inductor' goals left the buck at 44 mV worst-case ripple against a 50 mV limit. Aim for the middle of the limit, or give the user the size-vs-margin choice.

• **Make long finishes cheaper to wait for:** the buck needed 13 (script) and 23 (Haiku) identical 'still running' polls. Return an estimated finish time and tell the model to wait that long, or report progress instead.

• **Say 'not buildable at this tolerance' up front:** when the spec window is narrower than the part tolerance (RC ±2% with 5% capacitors), say so before optimising.

Limits of this benchmark

• Three circuits, one run per model and arm; runs are not repeated, so luck is not averaged out.

• Simulation only (ngspice 42). The buck ripple needs a 2 ns time step to be honest: at 20 ns SpiceMCP's buck reads 180 mV. The raw-simulator buck only runs at 20 ns or 5 ns, so its ripple number is less certain.

• Buck yield is 30 boards per arm (90 runs at three temperatures): 30/30 means at least 90% with 95% confidence. RC and amplifier use 200 boards.

• Tokens are counted with an older public tokenizer and exclude the tool harness's own system prompt, which is the same for every arm.